SEMT30008 Week 19 Lab 2: Large Language Model Foundation —— Transformer

Course Introduction & Lab Objectives

Welcome to the LLM Foundations Lab! This 2-hour interactive session is based on lecture slides. Instead of just reading about Transformers, you are going to build their core mathematical components from scratch using PyTorch. In this lab, you will complete 6 parts:

- NLP Preprocessing & Word2Vec (Slides 9-24)
- Positional Encoding (Slides 36-38)
- The Core of the Encoder: Self-Attention (Slides 49-56)
- Multi-Head Attention Intuition (Slides 57-63)
- Masked & Cross Attention in the Decoder (Slides 64-73)
- Challenge Section: 10 Interactive Exercises (Test your knowledge)

⚙ Part 0: Environment Setup

Let's import PyTorch and visualization libraries. Run the cell below.

In [ ]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import math
import matplotlib.pyplot as plt

# Set printing precision to align exactly with the lecture slides
torch.set_printoptions(precision=2, sci_mode=False)
print("✅ Libraries imported successfully! You are ready to begin.")
```

```
✅ Libraries imported successfully! You are ready to begin.
```

🧠 Part 3: Encoder - Self-Attention (Slides 49-56)

This is the core of the Transformer. We will calculate the exact matrices shown in Slides 49-56 for the sentence "Thinking Machines". We will manually multiply the Query, Key, and Value matrices.

In [ ]:
```
#[Colab Code Block 4]
print("=== Recreating Slides 49 to 56: Self-Attention ===")

# Slide 50: The learned representations for "Thinking" (Row 1) and "Machines"
Q = torch.tensor([[1.0, 0.0, 2.0], [0.0, 1.0, 1.0]]) # Query for "Thinking", Q
```

```
K = torch.tensor([[2.0, 0.0, 1.0], [0.0, 2.0, 1.0]]) # Key for "Thinking", Key

V = torch.tensor([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]) # Value for "Thinking", V

# Todo Slide 53: Calculate Dot Products (Scores = Q * K^T)
scores =
# End

print("\n1. Raw Scores (Q * K^T):\n", scores) # Expected: [[4, 2], [1, 3]]

# Slide 54: Scaling
# Slide assumes sqrt(d_k) = 2. Scaling prevents vanishing gradients in softmax
sqrt_dk = 2.0
scaled_scores = scores / sqrt_dk
print("\n2. Scaled Scores:\n", scaled_scores) # Expected: [[2.0, 1.0], [0.5, 1

# Slide 55: Apply Softmax
weights = F.softmax(scaled_scores, dim=-1)
print("\n3. Attention Weights (Softmax):\n", weights) # Expected approx: [[0.7

# Todo Slide 56: Multiply by Values (Z = Weights * V)
Z =
# End

print("\n4. Final Output Z:\n", Z)
# Look at Row 1 (Thinking): It is 0.73 * [1,2,3] + 0.27 * [4,5,6]
print("💡 Intuition: 'Thinking' has successfully absorbed 27% of the contextua
```

🎭 Part 4: Multi-Head Attention Intuition (Slides 57-63)

Why is one head not enough? As Slide 57 explains: Head 1 (Grammar): Focuses on Subject-Verb relationships. Head 2 (Meaning): Focuses on Reference ("it" refers to "animal"). In practice, we take a large vector and split it into smaller "heads".

In [ ]:
```
#[Colab Code Block 5]
# Let's say our word embedding is 8-dimensional
word_embedding = torch.tensor([[1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0]])

num_heads = 2
d_model = word_embedding.shape[1] # 8
d_k = d_model // num_heads        # 4 dimensions per head

print(f"Original Embedding shape: {word_embedding.shape}")

# Split into 2 heads
# Reshape from [1, 8] to[1, 2 heads, 4 dims]
heads = word_embedding.view(1, num_heads, d_k)

print(f"\nHead 1 (Focuses on grammar): \n{heads[0, 0, :]}")
print(f"Head 2 (Focuses on meaning): \n{heads[0, 1, :]}")
```

```
print("\n💡 After parallel self-attention computation, these heads are concate
```

🛡️ Part 5: Decoder Masking & Cross-Attention (Slides 65-72)

In the Decoder, the model generates words one by one. It is forbidden to look at future words. We apply a Mask (Slide 67). Next, it queries the Encoder's memory using Cross-Attention (Slide 70).

In [ ]:
```
# [Colab Code Block 6]
print("=== Decoder: Masked Attention (Slides 66-68) ===")
# Suppose we are generating a sequence of 3 tokens. Here is the raw score matr
E = torch.tensor([[1.0, 2.0, 3.0],
                  [2.0, 4.0, 6.0],[3.0, 6.0, 9.0]])

# Create a Look-ahead Mask (Upper triangle filled with -inf)
mask = torch.triu(torch.ones(3, 3), diagonal=1).bool()
M = torch.zeros(3, 3).masked_fill(mask, float('-inf'))

print("Mask Matrix:\n", M)
print("\nMasked Scores (E + M):\n", E + M)

# Softmax effectively turns -inf into 0% probability
attention_probs = F.softmax(E + M, dim=-1)
print("\nFinal Probabilities (Future is completely hidden!):\n", attention_prc


print("\n=== Decoder: Cross-Attention (Slides 70-72) ===")
# Encoder generated memory for "The cat"
K_enc = torch.tensor([[1.0, 0.0],[0.0, 1.0]])
V_enc = K_enc.clone()

# Decoder is currently looking for the next word after generating "Le"
Q_dec = torch.tensor([[0.0, 2.0]])

# Todo Calculate alignment
cross_scores = torch.matmul(???)
# End

print("Raw Cross-Attention Score (Alignment with 'The' and 'cat'):", cross_scc

cross_weights = F.softmax(cross_scores, dim=-1)
print("Softmax Weights:", cross_weights) # Result: [0.12, 0.88]

context_vector = torch.matmul(cross_weights, V_enc)
print("Context Vector pulled from Encoder:", context_vector)
print("💡 The decoder has realized it needs to pay 88% attention to 'cat' to g
```

Exercise 6: Manual Attention Score

Task: Calculate the dot product score (unscaled) between (Q_vec = [1.5, 2.0]) and (K_vec = [0.5, -1.0]).

In [ ]:
```
Q_v = torch.tensor([1.5, 2.0])
K_v = torch.tensor([0.5, -1.0])

# --- YOUR CODE HERE ---
score =
# ----------------------

print("Ex 6 Dot Product Score:", score.item())
assert score.item() == -1.25, "Exercise 6 Failed!"
print("✅ Exercise 6 Passed!")
```

Exercise 7: Row-wise Softmax

Task: You have a scaled score matrix of shape [2, 2]. Apply F.softmax. Be careful to apply it on the correct dimension (dim = -1) so that each row sums to 1.

In [ ]:
```
scaled_m = torch.tensor([[0.0, 0.0],[100.0, 0.0]]) # Very high score for the f

# --- YOUR CODE HERE ---
attn_weights =
# ----------------------

print("Ex 7 Softmax Weights:\n", attn_weights)
assert torch.allclose(attn_weights[0], torch.tensor([0.5, 0.5])), "Exercise 7
assert torch.allclose(attn_weights[1], torch.tensor([1.0, 0.0])), "Exercise 7
print("✅ Exercise 7 Passed!")
```

Exercise 8: Creating the Mask

Task: Create a look-ahead mask for a sequence of length 4. The upper triangle (excluding the diagonal) should be filled with float('-inf') and the rest with 0.0.

In [ ]:
```
length = 4

# --- YOUR CODE HERE ---
target_mask =
bool_mask =
target_mask.masked_fill_(bool_mask, float('-inf'))
# ----------------------

print("Ex 8 Mask:\n", target_mask)
assert target_mask[0, 1].item() == float('-inf'), "Exercise 8 Failed!"
assert target_mask[1, 1].item() == 0.0, "Exercise 8 Failed!"
```

```
print("✅ Exercise 8 Passed!")
```

Exercise 9: Final Context Vector

Task: Your softmax weights are ([0.1, 0.9]) and your Value matrix (V) is ([[10, 20], [100, 200]]). Calculate the final attention output (Z).

In [ ]:
```
w_tensor = torch.tensor([0.1, 0.9])
v_tensor = torch.tensor([[10.0, 20.0],[100.0, 200.0]])

# --- YOUR CODE HERE ---
# Use torch.matmul
Z_output =
# ----------------------

print("Ex 9 Z Output:", Z_output)
assert torch.allclose(Z_output, torch.tensor([91.0, 182.0])), "Exercise 9 Fail
print("✅ Exercise 9 Passed!")
```

Exercise 10: Put It All Together (Self-Attention Function)

Task: Write a complete function that takes (Q), (K), (V) and a boolean `use_mask` parameter. It should compute the full Self-Attention output (Z). Assume scaling factor (\sqrt{d_k} = 2.0).

In [ ]:
```
#[Colab Code Block 16 - Exercise 10]
def calculate_attention(Q, K, V, use_mask=False):
    # --- YOUR CODE HERE ---
    # 1. Dot product
    scores =

    # 2. Scale
    scaled_scores =

    # 3. Mask (Optional)
    if use_mask:
        seq_len = Q.size(0)
        mask =
        scaled_scores =

    # 4. Softmax
    weights = F.softmax(scaled_scores, dim=-1)

    # 5. Value multiplication
    Z = torch.matmul(weights, V)
    return Z
    # ----------------------

# Test using our matrices from Part 3
Q_test = torch.tensor([[1.0, 0.0, 2.0],[0.0, 1.0, 1.0]])
```

```
K_test = torch.tensor([[2.0, 0.0, 1.0], [0.0, 2.0, 1.0]])
V_test = torch.tensor([[1.0, 2.0, 3.0],[4.0, 5.0, 6.0]])

out_unmasked = calculate_attention(Q_test, K_test, V_test, use_mask=False)
out_masked = calculate_attention(Q_test, K_test, V_test, use_mask=True)

print("Unmasked Output:\n", out_unmasked)
print("\nMasked Output:\n", out_masked)

# The masked output for row 0 should perfectly match V[0] because it cannot lo
assert torch.allclose(out_masked[0], V_test[0]), "Exercise 10 Failed!"
print("✅ Exercise 10 Passed! Congratulations, you have mastered the Transform
```

In [ ]: